

XI BOB. MURAKKAB NUMPY (ADVANCED NUMPY)

NumPy-ning murakkab (Advanced) bo'limiga xush kelibsiz! Agar siz massivlar bo'yicha asosiy bilimlarni egallagan bo'lsangiz, ma'lumotlar bilan ishlashni yanada tezroq va osonroq qiladigan kuchli vositalarni o'rganishga tayyorsiz.

11.1. NumPy Massivlarini Yoyish (Broadcasting)

NumPy-da **Broadcasting** (translyatsiya) — bu o'lchamlari turlicha bo'lgan massivlar ustida ularning shaklini o'zgartirmasdan turib arifmetik amallarni bajarish imkonini beruvchi mexanizmdir. U kichikroq massivning qiymatlarini kerakli o'lchamlar bo'ylab nusxalash orqali uni kattaroq massiv shakliga avtomatik ravishda moslashtiradi. Bu elementma-element bajariladigan amallarni samaraliroq qiladi, chunki xotira sarfi kamayadi va qo'shimcha sikllarga (loops) ehtiyoj qolmaydi.

Keling, misol ko'raylik:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
x = 10
print(a + x)
```

```
[[11 12 13]
 [14 15 16]]
```

Izoh:

- NumPy `x` skalyar qiymatini `a` massivining shakliga mos kelishi uchun "kengaytiradi".
- `a + x` amali `a` massivining har bir elementiga 10 sonini qo'shib chiqadi.

Broadcasting qoidalari: Ikki massiv bir-biriga mos kelishi (broadcast qilinishi) uchun quyidagi shartlardan biri bajarilishi kerak:

1. Massivlarning mos o'lchamlari teng bo'lishi kerak.
2. o'lchamlardan biri **1** ga teng bo'lishi kerak.

Agar bu shartlar bajarilmasa, NumPy `ValueError: operands could not be broadcast together` xatoligini qaytaradi.

NumPy-da Broadcasting (translyatsiya) qanday ishlaydi: Broadcasting ikki massivning amallarni bajarish uchun o‘zaro mos kelishini aniqlashda bir nechta aniq qoidalarini qo‘llaydi:

- **o‘lchamlarni tekshirish:** Massivlar bir xil miqdordagi o‘lchamlarga ega ekanligini yoki o‘lchamlarni kengaytirish mumkinligini tekshiradi.
- **o‘lchamlarni to‘ldirish (Padding):** Agar massivlar turli o‘lchamlarga ega bo‘lsa, kichikroq massiv chap tomondan birliklar bilan to‘ldiriladi.
- **Shakl mosligi (Compatibility):** Ikki o‘lcham o‘zaro mos hisoblanadi, agar ular teng bo‘lsa yoki ulardan biri 1 ga teng bo‘lsa.

Agar bu shartlar bajarilmasa, NumPy **ValueError** xatoligini qaytaradi. Quyida broadcasting-ga oid turli misollarni ko‘rishingiz mumkin:

1-misol: Skalyar qiymatni 1D massivga broadcast qilish

Bu yerda `[1, 2, 3]` qiymatlariga ega `arr` massivi yaratiladi va broadcasting yordamida massivning har bir elementiga skalyar 1 qiymati qo‘shiladi.

```
import numpy as np
arr = np.array([1, 2, 3])
res = arr + 1
print(res)
```

`[2 3 4]`

2-misol: 1D massivni 2D massivga broadcast qilish

Ushbu misolda `a` (1D massiv) qanday qilib `b` (2D massiv) ga qo‘shilishi ko‘rsatilgan. NumPy elementma-element qo‘shishni bajarish uchun 1D massivni 2D massiv qatorlari bo‘ylab avtomatik ravishda kengaytiradi.

```
a = np.array([2, 4, 6])
b = np.array([[1, 3, 5], [7, 9, 11]])
res = a + b
print(res)
```

```
[[ 3  7 11]
 [ 9 13 17]]
```

Izoh:

- `a` massivi `(3,)` shakliga ega, `b` massivi esa `(2, 3)` shaklida.
- NumPy shakllar mos kelishi uchun `a` massivini `b` ning ikkala qatori bo'ylab avtomatik ravishda takrorlaydi.
- Keyin elementlarni o'rni bo'yicha qo'shadi:
`[1, 3, 5] + [2, 4, 6] = [3, 7, 11]` va
`[7, 9, 11] + [2, 4, 6] = [9, 13, 17]`.

3-misol: Shartli amallarda Broadcasting

Ushbu misol massivdagi har bir yoshni tekshiradi va `np.where()` funksiyasidan foydalangan holda ularga "Adult" (Katta) yoki "Minor" (Voyaga yetmagan) yorlig'ini biriktiradi.

```
a = np.array([12, 24, 35, 45, 60, 72])
b = np.array(["Adult", "Minor"])

# a > 18 sharti har bir qiymatni birdaniga tekshirib,
# mantiqiy (Boolean) massiv yaratadi (broadcasting).
res = np.where(a > 18, b[0], b[1])

print(res)

['Minor' 'Adult' 'Adult' 'Adult' 'Adult' 'Adult']
```

Izoh:

- `a > 18` amaliy broadcasting yordamida har bir element uchun `True` yoki `False` qiymatlarini qaytaradi.
- `np.where()` hech qanday sikl (loop) ishlatmasdan, `True` uchun "Adult", `False` uchun esa "Minor" qiymatini tanlab oladi.
- Natijada har bir yosh to'g'ri belgilangan massiv hosil bo'ladi.

4-misol: Matrisani ko'paytirishda Broadcasting-dan foydalanish

Ushbu misolda 2D matrisaning har bir elementi broadcast qilingan vektorning mos keluvchi elementi bilan ko'paytiriladi.

```
m = np.array([[1, 2], [3, 4]])
v = np.array([10, 20])

# v vektori m matrisasining har bir qatori bo'ylab broadcast qilinadi.
```

```
res = m * v
```

```
print(res)
```

```
[[10 40]  
 [30 80]]
```

Izoh:

- `v` vektori `m` matrisasining har bir qatoriga mos kelishi uchun avtomatik ravishda kengaytiriladi.
- Ko‘paytirish amali elementma-element (element-wise), sikllarsiz amalga oshiriladi.
- Natija matrisaning masshtablangan (har bir ustun mos skalyarga ko‘paytirilgan) ko‘rinishidir.

5-misol: ma’lumotlarni Broadcasting yordamida masshtablash (Scaling)

Keling, hayotiy misolni ko‘rib chiqamiz: oziq-ovqat tarkibidagi yog‘lar, oqsillar va uglevodlar miqdoriga qarab ularning umumiy kaloriyasini hisoblashimiz kerak. Har bir ozuqa moddasi bir gramm uchun o‘ziga xos kaloriya qiymatiga ega:

- **Yog‘lar:** bir grammda 9 kaloriya (CPG)
- **Oqsillar:** 4 CPG
- **Uglevodlar:** 4 CPG

Ushbu misolda chap tomondagi jadval oziq-ovqat mahsulotlarini va ulardagi yog‘, oqsil hamda uglevod grammlarini ko‘rsatadi. `[9, 4, 4]` massivi esa mos ravishda ushbu moddalar uchun kaloriya koeffitsientlarini ifodalaydi. Ushbu massiv asl ma’lumotlar o‘lchamiga mos kelishi uchun broadcast qilinadi.

Broadcasting jarayonida koeffitsientlar massivi asl ma’lumotlarning har bir qatori bilan elementma-element ko‘paytiriladi. Natijada paydo bo‘lgan o‘ng tomondagi jadval har bir mahsulotdagi aniq bir ozuqa moddasining kaloriya miqdorini ko‘rsatadi.

```
# Oziq-ovqat ma’lumotlari: [Yog‘, Oqsil, Uglevod] grammlarda  
fd = np.array([ [0.8, 2.9, 3.9],  
                [52.4, 23.6, 36.5],  
                [55.2, 31.7, 23.9],
```

```

[14.4, 11.0, 4.9] ]))

# Har bir gramm uchun kaloriya qiymatlari
cpg = np.array([9, 4, 4])

# Broadcasting orqali ko'paytirish
res = fd * cpg

print(res)

[[ 7.2 11.6 15.6]
 [471.6 94.4 146. ]
 [496.8 126.8 95.6]
 [129.6 44. 19.6]]

```

Izoh:

- `cpg` (9, 4, 4) massivi `fd` matrisasining har bir qatori bo'ylab broadcast qilinadi.
- Har bir ozuqa moddasining grammi uning kaloriya qiymatiga ko'paytiriladi.
- Natija — har bir oziq-ovqat mahsuloti uchun yog'lar, oqsillar va uglevodlardan keladigan kaloriyalar hissasini ko'rsatuvchi matrisa.

6-misol: Bir nechta joylardagi harorat ma'lumotlarini tuzatish

Faraz qiling, sizda bir nechta shaharlardagi kunlik harorat ko'rsatkichlarini aks ettiruvchi 2D massiv bor va siz har bir shaharning harorat ma'lumotlariga o'ziga xos tuzatish koeffitsientini qo'llamoqchisiz.

```

# Shaharlar bo'yicha kunlik haroratlar
temp = np.array([ [30, 32, 34, 33, 31],
                  [25, 27, 29, 28, 26],
                  [20, 22, 24, 23, 21] ])

# Har bir shahar uchun tuzatish koeffitsienti
corr = np.array([1.5, -0.5, 2.0])

# corr[:, None] 1D massivni ustun vektoriga aylantiradi
res = temp + corr[:, None]

print(res)

[[31.5 33.5 35.5 34.5 32.5]
 [24.5 26.5 28.5 27.5 25.5]
 [22.  24.  26.  25.  23. ]]

```

Izoh:

- `corr[:, None]` amali 1D massivni (3, 1) shaklidagi ustun vektoriga aylantiradi.
- NumPy ushbu vektorni `temp` matrisasining har bir ustuni bo'ylab broadcast qiladi.
- Har bir shaharning harorati unga mos keladigan tuzatish koeffitsienti yordamida o'zgaradi.

7-misol: Tasvir ma'lumotlarini normallashtirish (Normalization)

Normallashtirish tasvirga ishlov berish va mashinali o'qitish kabi sohalarda juda muhim, chunki u:

- **ma'lumotlarni markazlashtiradi:** o'rtacha qiymatni (mean) ayirish orqali xususiyatlarning o'rtacha qiymati nol bo'lishini ta'minlaydi.
- **Masshtablaydi:** Standart og'ishga (std) bo'lish orqali xususiyatlarning dispersiyasini (unit variance) bir xillashtiradi.
- Gradient tushishi (gradient descent) kabi algoritmlarning barqarorligi va tezligini oshiradi.

Keling, broadcasting normallashtirishni qanchalik soddalashtirishini ko'ramiz:

```
img = np.array([ [100, 120, 130],
                 [90, 110, 140],
                 [80, 100, 120] ])

# Har bir ustun uchun o'rtacha qiymat va standart og'ish
m = img.mean(axis=0)
s = img.std(axis=0)

# Broadcasting yordamida normallashtirish
res = (img - m) / s

print(res)
```

```
[[ 1.22474487  1.22474487  0.
   0.
   0.
  -1.22474487]
 [ 0.
   0.
  -1.22474487]
 [-1.22474487 -1.22474487 -1.22474487]]
```

Izoh:

- `m` va `s` — bu 1D massivlar (har bir ustun uchun o'rtacha va standart og'ish).
- NumPy ularni `img` matrisasining barcha qatorlari bo'ylab broadcast qiladi.
- `(img - m)` amali ma'lumotlarni markazlashtiradi, `s` ga bo'lish esa ularni masshtablab, normallashtirilgan qiymatlarni beradi.

8-misol: Mashinali o'qitishda ma'lumotlarni markazlashtirish (Centering)

Ma'lumotlarni markazlashtirish mashinali o'qitishning ko'plab ish jarayonlarida muhim bosqich hisoblanadi. Broadcasting har bir xususiyatdan (feature) uning o'rtacha qiymatini ayirish orqali ma'lumotlarni samarali markazlashtirishga yordam beradi. Ushbu misol NumPy broadcasting yordamida har bir xususiyatni o'rtacha qiymatni ayirish orqali markazlashtiradi.

```
data = np.array([ [10, 20],
                  [15, 25],
                  [20, 30] ])

# Har bir ustun (xususiyat) bo'yicha o'rtacha qiymatni hisoblash
m = data.mean(axis=0)

# O'rtacha qiymatni ayirish (Broadcasting orqali)
res = data - m

print(res)
```

```
[[ -5. -5.]
 [  0.  0.]
 [  5.  5.]]
```

Izoh:

- `m` — bu har bir ustunning o'rtacha qiymatini o'z ichiga olgan 1D massiv (bu yerda `[15.0, 25.0]`).
- NumPy ushbu `m` massivini `data` matrisasining barcha qatorlari bo'ylab broadcast qiladi.
- Uni ayirish natijasida har bir xususiyat (ustun) nol atrofida markazlashadi.

Ma'lumotlarni markazlashtirish algoritmlarning (masalan, PCA - Asosiy komponentlar tahlili) to'g'ri ishlashi uchun zarur, chunki u ma'lumotlar

to'plamidagi o'zgaruvchanlikni (variance) o'rtacha qiymatdan xoli ravishda tahlil qilish imkonini beradi.

11.2. NumPy-da vektorlashtirish (Vectorization)

NumPy-da **Vektorizatsiya (Vectorization)** — bu amallarni ochiq sikllardan (loops) foydalanmasdan butun massivlarga nisbatan qo'llashni anglatadi. Ushbu amallar ichki tomondan tezkor C/C++ tillarida optimallashtirilgan bo'lib, bu raqamli hisob-kitoblarni yanada samaraliroq va yozish uchun osonroq qiladi.

Nima uchun Vektorizatsiya muhim?

Vektorizatsiya quyidagi sabablarga ko'ra muhim ahamiyatga ega:

- **Samaradorlikni oshiradi:** Python darajasidagi sikllarni yo'qotadi va quyi darajadagi (low-level) tezkor dasturlash tillaridan foydalanadi.
- **Toza kod yaratadi:** Kamroq qatorlar, texnik xizmat ko'rsatish osonroq.
- **Yaxshi masshtablanadi:** Katta hajmdagi ilmiy ma'lumotlar va mashinali o'qitish yuklamalarini samarali bajara oladi.

Vektorizatsiyaga oid misollar

1-misol: Har bir elementga son qo'shish

Hech qanday sikllarsiz butun massiv bo'ylab elementma-element qo'shishni bajaradi, bu esa amalni tez va samarali qiladi.

```
a1 = np.array([2, 4, 6, 8, 10])
num = 2
res = a1 + num
print(res)
```

```
[ 4  6  8 10 12]
```

2-misol: Ikki massivni elementma-element qo'shish

Ikki NumPy massivining mos keluvchi elementlarini o'zaro qo'shadi.

```
a1 = np.array([1, 2, 3])
a2 = np.array([4, 5, 6])
res = a1 + a2
print(res)
```

```
[5 7 9]
```


Izoh: Python-da odatdagi ro'yxatlar (lists) bilan ishlaganda, har bir elementni qo'shish uchun `for` siklidan foydalanishga to'g'ri keladi. NumPy esa o'zining vektorlashtirilgan tabiati tufayli bu amalni bitta qator kod bilan va ancha yuqori tezlikda bajaradi. Bu nafaqat qo'shish, balki ko'paytirish, darajaga ko'tarish va matematik funksiyalar ($\sin$, $\cos$, $\log$) uchun ham amal qiladi.

3-misol: Elementma-element skalyar ko'paytirish

Massivning har bir elementini sikllar o'rniga tezkor vektorlashtirilgan massiv amallari yordamida o'zgaras songa ko'paytiradi.

```
a1 = np.array([1, 2, 3, 4])
res = a1 * 2
print(res)
```

```
[2 4 6 8]
```

4-misol: Massivlarda mantiqiy amallar

Taqqoslash kabi mantiqiy amallar massivlarga to'g'ridan-to'g'ri qo'llanilishi mumkin.

```
a1 = np.array([10, 20, 30])
res = a1 > 15
print(res)
```

```
[False  True  True]
```

Izoh: Elementma-element taqqoslashni bajaradi va qaysi elementlar 15 dan katta ekanligini ko'rsatuvchi mantiqiy (boolean) massiv qaytaradi.

5-misol: Vektorizatsiya yordamida matrisa amallari

NumPy skalyar ko'paytma (dot product) va matrisalarni ko'paytirish kabi vektorlashtirilgan matrisa amallarini `np.dot` va `@` operatori kabi funksiyalar orqali qo'llab-quvvatlaydi.

```
a1= np.array([[1, 2], [3, 4]])
a2 = np.array([[5, 6], [7, 8]])
res = np.dot(a1, a2)
print(res)
```

```
[[19 22]
 [43 50]]
```

Izoh: `a1` va `a2` matrisalari o'rtasida matrisa ko'paytmasini (dot product) bajaradi. Bu yerda har bir element qator va ustunlarning mos ravishda ko'paytmasi yig'indisi sifatida hisoblanadi (masalan, $1 \times 5 + 2 \times 7 = 19$).

6-misol: `np.vectorize()` yordamida maxsus funksiyalarni qo'llash

`np.vectorize` metodi massivning har bir elementiga maxsus (user-defined) funksiyani elementma-element qo'llash imkonini beradi. Masalan, har bir element uchun $x^2 + 2x + 1$ ifodasini hisoblash.

```
a1 = np.array([1, 2, 3, 4])
# Maxsus funksiyani vektorizatsiya qilish
vec = np.vectorize(lambda x: x**2 + 2*x + 1)
res = vec(a1)
print(res)
```

```
[ 4  9 16 25]
```

Izoh: NumPy-ning vektorlashtirilgan arifmetikasi yordamida `a1` massivdagi har bir x qiymati uchun ko'rsatilgan matematik amal bajariladi. Garchi `np.vectorize` kodni yozishni qulay qilsa-da, u asosan qulaylik uchun xizmat qiladi va har doim ham NumPy-ning o'z ichki (native) funksiyalaridek tez bo'lmasligi mumkin.

7-misol: Vektorlashtirilgan agregat amallari

`sum` (yig'indi), `mean` (o'rtacha qiymat), `max` (maksimal qiymat) kabi amallar NumPy-da optimallashtirilgan bo'lib, ular elementlar bo'ylab an'anaviy Python sikllaridan foydalanishga qaraganda ancha tezroq ishlaydi.

```
a1 = np.array([1, 2, 3])
r1 = a1.sum()
r2 = a1.mean()
print(r1)
print(r2)
```

```
6
2.0
```

Izoh: NumPy-ning vektorlashtirilgan agregat funksiyalari yordamida `a1` massivdagi barcha elementlarning yig'indisi (`r1`) va o'rtacha qiymati (`r2`) hisoblanadi. Bu amallar massiv hajmi juda katta bo'lganda ham soniyaning kichik qismlarida bajariladi.

Samaradorlikni solishtirish: Sikl (Loop) va Vektorizatsiya

Katta ma'lumotlar to'plami bilan ishlashda unumdorlik juda muhim ahamiyatga ega. Pandas va NumPy kutubxonalarida vektorizatsiya deyarli har doim qo'lda yozilgan Python sikllaridan tezroq ishlaydi. Buning sababi shundaki, vektorlashtirilgan amallar ichki tomondan optimallashtirilgan C

tilidagi kodda bajariladi, Python sikllari esa Python interfeysida qatorma-qator (ancha sekin) ishlaydi.

Misol: Katta NumPy massivini yaratamiz va bir xil amalni (har bir elementni 2 ga ko'paytirish) ikki xil usulda bajaramiz:

1. **For sikli** (Python darajasida)
2. **Vektorlashtirilgan amal** (NumPy darajasida)

```
import numpy as np
import time

# 1 million elementdan iborat massiv yaratish
arr = np.arange(1_000_000)

# sikl yordamida (Loop)
t1 = time.time()
loop_res = [x * 2 for x in arr]
t2 = time.time()

# Vektorizatsiya yordamida (Vectorized)
t3 = time.time()
vec_res = arr * 2
t4 = time.time()

print("Sikl vaqti (Loop Time):", t2 - t1)
print("Vektorizatsiya vaqti (Vectorized Time):", t4 - t3)
```

Sikl vaqti (Loop Time): 0.06891179084777832

Vektorizatsiya vaqti (Vectorized Time): 0.002093076705932617

Izoh:

- `arr = np.arange(1_000_000)` : 1 million sondan iborat NumPy massivini yaratadi.
- **Sikl usuli:** `[x * 2 for x in arr]` har bir elementni Python-da birma-bir qayta ishlaydi, bu sekin jarayon. `t2 - t1` sikl qancha vaqt olganini o'lchaydi.
- **Vektorlashtirilgan usul:** `arr * 2` NumPy ichidagi tezkor va optimallashtirilgan C-darajasidagi amallardan foydalanadi. `t4 - t3` vektorizatsiya qanchalik tez ekanligini ko'rsatadi.

Xulosa: Vektorizatsiya sezilarli darajada tezroq, chunki amallar Python-ning sekin, elementma-element ishlaydigan siklida emas, balki

optimallashtirilgan quyi darajadagi (low-level) kodda amalga oshiriladi.

11.3. Universal Funksiyalar (ufuncs) — NumPy-ning hisoblash mexanizmi

NumPy **ufuncs** (universal functions — universal funksiyalar) — bu NumPy massivlari ustida elementma-element amallarni bajaradigan tezkor, vektorlashtirilgan funksiyalardir. Ular yuqori darajada optimallashtirilgan bo‘lib, broadcasting va ma’lumotlar turlarini avtomatik boshqarish kabi xususiyatlarni qo‘llab-quvvatlaydi. NumPy arifmetika, trigonometriya, statistika va boshqalar uchun ko‘plab ufunc'larni o‘z ichiga oladi va ular optimallashtirilgan **C tilida** yozilgani uchun juda tez ishlaydi.

Nima uchun ufuncs ishlatiladi?

- **Tezkor vektorlashtirilgan amallar:** Ufuncs hisob-kitoblarni birdaniga butun massivga qo‘llaydi, bu esa ularni Python sikllaridan ancha tezroq qiladi.
- **Avtomatik broadcasting va turlarni boshqarish:** Ular massiv shakllarini avtomatik moslashtiradi va ma’lumot turlarini ichki konvertatsiya qiladi, bu esa xatolarni kamaytiradi.
- **Toza va samarali kod:** Murakkab raqamli vazifalarni qisqa, tushunarli kodga aylantiradi, bu esa katta ma’lumotlar to‘plamlarida yaxshi ishlaydi.

NumPy-da asosiy universal funksiyalar (ufunc)

1. Trigonometrik funksiyalar

NumPy massivlar ustida elementma-element ishlaydigan bir nechta trigonometrik funksiyalarni taqdim etadi. Burchaklar **radianlarda** bo‘lishi kerak, shuning uchun daraja qiymatlarini `np.deg2rad()` yordamida o‘tkazish lozim.

NumPy-dagi keng tarqalgan trigonometrik ufunc'lar:

Funksiya	Tavsif
<code>np.sin</code> , <code>np.cos</code> , <code>np.tan</code>	Burchaklarning sinusi, kosinusi va tangensini hisoblash

Funksiya	Tavsif
<code>np.arcsin</code> , <code>np.arccos</code> , <code>np.arctan</code>	Teskari sinus, kosinus va tangensni hisoblash
<code>np.sinh</code> , <code>np.cosh</code> , <code>np.tanh</code>	Giperbolik sinus, kosinus va tangensni hisoblash
<code>np.deg2rad</code>	Darajalarni radianlarga o'tkazish
<code>np.rad2deg</code>	Radianlarni darajalarga o'tkazish
<code>np.hypot</code>	To'g'ri burchakli uchburchakning gipotenuzasini hisoblash

Misol: Ushbu misolda sinus, teskari sinus, giperbolik sinus va gipotenuza hisoblash ko'rsatilgan.

```
# Burchaklar darajada
angles = np.array([0, 30, 45, 60, 90])
rad = np.deg2rad(angles) # darajalarni radianlarga o'tkazish

# Burchaklarning sinusi
sin_vals = np.sin(rad)
print("Sinus qiymatlari:", sin_vals)

# Teskari sinus darajalarda
inv_sin = np.rad2deg(np.arcsin(sin_vals))
print("Teskari sinus (darajalarda):", inv_sin)

# Giperbolik sinus
sinh_vals = np.sinh(rad)
print("Giperbolik sinus:", sinh_vals)

# To'g'ri burchakli uchburchak gipotenuzasi
hyp = np.hypot(3, 4)
print("Gipotenuza:", hyp)
```

```
Sinus qiymatlari: [0.          0.5         0.70710678 0.8660254  1.
]
```

```
Teskari sinus (darajalarda): [ 0. 30. 45. 60. 90.]
```

```
Giperbolik sinus: [0.          0.54785347 0.86867096 1.24936705 2.30129
89 ]
```

```
Gipotenuza: 5.0
```

Izoh:

- `np.deg2rad()` darajalarni radianlarga aylantiradi.
- `np.sin()` har bir element uchun sinusni hisoblaydi.
- `np.arcsin()` teskari sinusni (arksinus) hisoblaydi; `np.rad2deg()` uni qaytadan darajaga o'tkazadi.
- `np.hypot(a, b)` tomonlari a va b bo'lgan to'g'ri burchakli uchburchakning gipotenuzasini ($\sqrt{a^2 + b^2}$) hisoblaydi.

2. Statistik funksiyalar

NumPy o'rtacha qiymat, median, dispersiya va diapazon kabi xususiyatlarni hisoblash uchun bir nechta statistik funksiyalarni taqdim etadi. Ushbu funksiyalar elementma-element va belgilangan o'qlar (axes) bo'ylab ishlaydi, bu esa massivlarni tahlil qilishni tez va samarali qiladi.

NumPy-dagi keng tarqalgan statistik ufunc'lar:

Funksiya	Tavsif
<code>np.amin</code> , <code>np.amax</code>	Massivning yoki ma'lum bir o'q bo'yicha eng kichik va eng katta qiymati
<code>np.ptp</code>	Massiv qiymatlari diapazoni (maksimal – minimal)
<code>np.percentile(a, p)</code>	Massiv qiymatlarining p-chi persentili
<code>np.mean</code>	ma'lumotlarning o'rtacha arifmetik qiymatini hisoblash
<code>np.median</code>	ma'lumotlarning medianasini hisoblash
<code>np.std</code>	Standart og'ishni hisoblash
<code>np.var</code>	Dispersiyani (variance) hisoblash
<code>np.average</code>	o'rtacha qiymatni hisoblash

Misol: Ushbu misol talabalar vazni massivida umumiy statistik hisob-kitoblarni qanday bajarishni ko'rsatadi.

```

weights = np.array([50.7, 52.5, 50, 58, 55.63, 73.25, 49.5, 45])

# Minimal va Maksimal qiymat
print("Min va Max:", np.amin(weights), np.amax(weights))

# Diapazon (Range)
print("Diapazon:", np.ptp(weights))

# 70-chi persentil
print("70-chi persentil:", np.percentile(weights, 70))

# o'rtacha qiymat (Mean)
print("o'rtacha qiymat:", np.mean(weights))

# Mediana
print("Mediana:", np.median(weights))

# Standart og'ish
print("Standart og'ish:", np.std(weights))

# Dispersiya (Variance)
print("Dispersiya:", np.var(weights))

# o'rtacha (Average)
print("o'rtacha:", np.average(weights))

```

```

Min va Max: 45.0 73.25
Diapazon: 28.25
70-chi persentil: 55.317
O'rtacha qiymat: 54.3225
Mediana: 51.6
Standart og'ish: 8.052773978574091
Dispersiya: 64.84716875
O'rtacha: 54.3225

```

Izoh:

- `np.amin()` va `np.amax()` eng kichik va eng katta qiymatlarni topadi.
- `np.ptp()` (peak-to-peak) diapazonni, ya'ni maksimal va minimal qiymat ayirmasini hisoblaydi.
- `np.percentile(weights, 70)` vaznlarning 70 foizi ushbu qiymatdan pastda joylashgan nuqtani topadi.
- `np.mean()` va `np.average()` massivning o'rtacha qiymatini hisoblaydi.

- `np.median()` o'rtadagi qiymatni qaytaradi.
- `np.std()` va `np.var()` ma'lumotlarning o'rtacha qiymatdan qanchalik tarqalganligini standart og'ish va dispersiya orqali o'lchaydi.

3. Bitli (Bitwise) funksiyalar

NumPy butun sonlarning ikkilik (binary) ko'rinishi ustida amallar bajarish uchun bitli funksiyalarni taqdim etadi. Bu funksiyalar massiv elementlarini bit darajasida boshqarish imkonini beradi.

NumPy-dagi keng tarqalgan bitli ufunc'lar:

Funksiya	Tavsif
<code>np.bitwise_and</code>	Elementma-element "VA" (AND) amali
<code>np.bitwise_or</code>	Elementma-element "YOKI" (OR) amali
<code>np.bitwise_xor</code>	Elementma-element "Inkor etuvchi YOKI" (XOR) amali
<code>np.invert</code>	Elementlarning bitlarini teskarisiga o'girish (NOT)
<code>np.left_shift</code>	Elementlar bitlarini chapga surish
<code>np.right_shift</code>	Elementlar bitlarini o'ngga surish

Misol: Ushbu misol butun sonlar massivlari ustida asosiy bitli amallarni ko'rsatadi.

```
even = np.array([0, 2, 4, 6, 8, 16, 32])
odd  = np.array([1, 3, 5, 7, 9, 17, 33])

# Bitli AND, OR, XOR
print("AND:", np.bitwise_and(even, odd))
print("OR :", np.bitwise_or(even, odd))
print("XOR:", np.bitwise_xor(even, odd))

# Bitli NOT (Invert)
print("Invert:", np.invert(even))

# Bitlarni surish (Shifts)
print("Chapga surish :", np.left_shift(even, 1))
print("O'ngga surish:", np.right_shift(even, 1))
```



```
AND: [ 0  2  4  6  8 16 32]
OR : [ 1  3  5  7  9 17 33]
XOR: [1 1 1 1 1 1 1]
Invert: [ -1  -3  -5  -7  -9 -17 -33]
Chapga surish : [ 0  4  8 12 16 32 64]
O'ngga surish: [ 0  1  2  3  4  8 16]
```

Izoh:

- `np.bitwise_and()` , `np.bitwise_or()` , `np.bitwise_xor()` funksiyalari bit darajasida mantiqiy amallarni bajaradi.
- `np.invert()` har bir elementning barcha bitlarini teskarisiga o'zgartiradi (0 ni 1 ga, 1 ni 0 ga).
- `np.left_shift()` va `np.right_shift()` har bir element bitlarini chapga yoki o'ngga suradi. Chapga surish amalda sonni 2 ning darajalariga ko'paytirishga, o'ngga surish esa bo'lishga tengdir.

11.4. NumPy kutubxonasida tasvirlar bilan ishlash (Working with Images in NumPy)

Tasvirga ishlov berish (Image Processing) kompyuter ko'rishi (computer vision) va tibbiy diagnostika kabi sohalarda raqamli tasvirlarni yaxshilash va tahlil qilish uchun ishlatiladi. Python-da **NumPy** tasvirlarni piksellar darajasida samarali boshqarish uchun massivlar sifatida ko'radi, **SciPy**'ning `ndimage` moduli esa filtrlash va transformatsiya qilish vositalarini taqdim etib, tezkor va yengil ishlov berish imkonini beradi.

O'rnatish

Boshlashdan avval kerakli kutubxonalar o'rnatilganligiga ishonch hosil qiling:

```
pip install numpy scipy matplotlib imageio scikit-image
```

- **NumPy**: Massivlar va matematik hisob-kitoblar bilan ishlaydi.
- **SciPy**: Murakkab ilmiy va matematik funksiyalarni taqdim etadi.
- **Matplotlib**: Grafiklar va vizualizatsiyalar yaratadi.
- **ImageIO**: Tasvir fayllarini o'qiydi va saqlaydi.
- **scikit-image**: Tasvirga ishlov berish va tahlil qilish uchun maxsus asboblarni to'plam.

Tasvirga ishlov berishda tasvirning har bir pikseli NumPy massividagi bitta elementga mos keladi. Rangli tasvirlar odatda uch o'lchamli massiv (balandlik, kenglik, rang kanallari - RGB) ko'rinishida bo'ladi.

Tasvirni ochish va ko'rsatish

Har qanday tasvirga ishlov berish vazifasida birinchi qadam tasvirni yuklash va uni vizuallashtirishdir. Buning uchun biz tasvirni o'qishda `imageio.v3` va uni ko'rsatishda `matplotlib` kutubxonalaridan foydalanamiz.

```
import imageio.v3 as iio
import matplotlib.pyplot as plt

# Tasvirni o'qiyamiz
img = iio.imread("images/raccoon.png")

# Tasvirni ko'rsatamiz
plt.imshow(img)
plt.axis('off') # Tozaroq ko'rinish uchun o'qlarni yashiramiz
plt.show()
```



Eslatma: Tasvir fayli Python skriptingiz bilan bir xil papkada bo'lishi kerak, aks holda unga bo'lgan nisbiy yoki to'liq yo'lni (path) ko'rsatishingiz lozim.

Izoh:

- `iio.imread()` : Tasvirni NumPy massiviga yuklaydi.
- `plt.imshow()` : Uni vizuallashtiradi.
- `plt.axis('off')` : Grafik atrofidagi koordinata o'qlarini olib tashlaydi.

Tasvirdan NumPy massivini yaratish

Tasvir, mohiyatiga ko'ra, ko'p o'lchamli NumPy massividir. Filtrlarni va o'zgartirishlarni qo'llash uchun uning shakli (shape) va ma'lumot turini (dtype) bilish muhimdir.

```
import imageio.v3 as iio
import numpy as np

img = iio.imread("images/raccoon.png")

print("Shakli (Shape):", img.shape)
print("ma'lumot turi (Data type):", img.dtype)
```

Shakli (Shape): (178, 240, 4)
Ma'lumot turi (Data type): uint8

Izoh:

- **Shakli (Shape):** Tasvirning tuzilishini tushunishga yordam beradi (masalan, $768 \times 1024 \times 3$ — bu RGB rangli tasvir uchun balandlik, kenglik va 3 ta rang kanalini anglatadi).
- **ma'lumot turi (Data type):** Odatda `uint8` bo'lib, u pikseller qiymati 0 dan 255 gacha ekanligini ko'rsatadi.

RAW faylini yaratish

`.raw` fayli tasvir sensori yoki matrisasidan olingan xom ikkilik (binary) ma'lumotlarni saqlaydi. Bu, ayniqsa, siqilmagan ma'lumotlar bilan ishlov berish zanjirlarida (image pipelines) juda foydali.

```
import imageio.v3 as iio
import numpy as np

# Tasvirni o'qiydiz
img = iio.imread("images/raccoon.png")

# Massiv ma'lumotlarini .raw formatida saqlaydiz
img.tofile("images/raccoon.raw")
```

Natija: `raccoon.raw` fayli saqlandi.

Izoh: `tofile()` funksiyasi tasvir piksellari ma'lumotlarini hech qanday sarlavha (header) yoki metadata qo'shmasdan, shunchaki ikkilik fayl sifatida saqlaydi. Bu quyi darajadagi (low-level) tasvirga ishlov berish uchun qulaydir.

RAW faylini ochish

`.raw` fayllari bilan ishlashda, ikkilik ma'lumotlarni qaytadan foydalanish mumkin bo'lgan NumPy massiviga aylantirish uchun `np.fromfile()` funksiyasidan foydalanamiz.

```
import imageio.v3 as iio
import numpy as np

# Asl tasvir shaklini bilish uchun uni o'qiydiz
orig = iio.imread("images/raccoon.png")
h, w, c = orig.shape

# RAW fayldan ma'lumotlarni o'qiydiz
flat = np.fromfile('images/raccoon.raw', dtype=np.uint8)

# ma'lumotlarni asl shakliga (balandlik x kenglik x kanallar) qaytaradi
img = flat.reshape((h, w, c))

print("Massiv shakli:", img.shape)
```

Massiv shakli: (178, 240, 4)

Natija: Massiv shakli: (768, 1024, 3) (qiymat rasmingizga qarab farq qilishi mumkin)

Izoh:

- `fromfile()` : Ikkilik ma'lumotlarni o'qiydi, lekin u ma'lumotlar qanday tuzilganini (shaklini) bilmaydi.
- `reshape()` : ma'lumotlarni vizuallashtirish uchun uni qo'lda asl o'lchamlariga qaytarish shart. Chunki RAW fayl ichida rasmni balandligi yoki kengligi haqida ma'lumot saqlanmaydi, u faqat uzun bir "tekis" (flat) raqamlar qatoridir.

Statistik ma'lumotlarni olish

Piksel intensivligining minimal, maksimal va o'rtacha qiymatlarini tushunish tasvirning yorqinligi (**brightness**), kontrasti (**contrast**) va gistogramma taqsimoti haqida muhim ma'lumotlarni beradi.

```
import imageio.v3 as iio
import numpy as np

# Tasvirni o'qiyamiz
img = iio.imread("images/raccoon.png")

# Statistik ko'rsatkichlarni chiqaramiz
print("Maksimal qiymat (Max):", img.max())
print("Minimal qiymat (Min):", img.min())
print("O'rtacha qiymat (Mean):", img.mean())
```

```
Maksimal qiymat (Max): 255
Minimal qiymat (Min): 0
O'rtacha qiymat (Mean): 145.67792602996255
```

Izoh:

- **Maksimal va minimal qiymatlar:** Tasvirning dinamik diapazonini va kontrastini ko'rsatadi. Masalan, agar `Max=255` va `Min=0` bo'lsa, tasvir to'liq rang diapazonidan foydalanmoqda.
- **o'rtacha qiymat (Mean):** Tasvirning umumiy yorqinligi haqida tasavvur beradi. Agar bu qiymat past bo'lsa (masalan, 50), tasvir qorong'u, agar yuqori bo'lsa (masalan, 200), tasvir juda yorug' hisoblanadi.

Ushbu statistik ma'lumotlar tasvirga ishlov berishdan oldin (masalan, normallashtirish yoki gistogrammani tekislashdan avval) ma'lumotlar sifatini baholash uchun juda foydalidir.

Tasvirni qirqish (Cropping)

Tasvirni qirqish NumPy massivini kesish (slicing) orqali tasvirning ma'lum bir qiziqarli sohasiga (**Region of Interest - ROI**) e'tibor qaratish imkonini beradi.

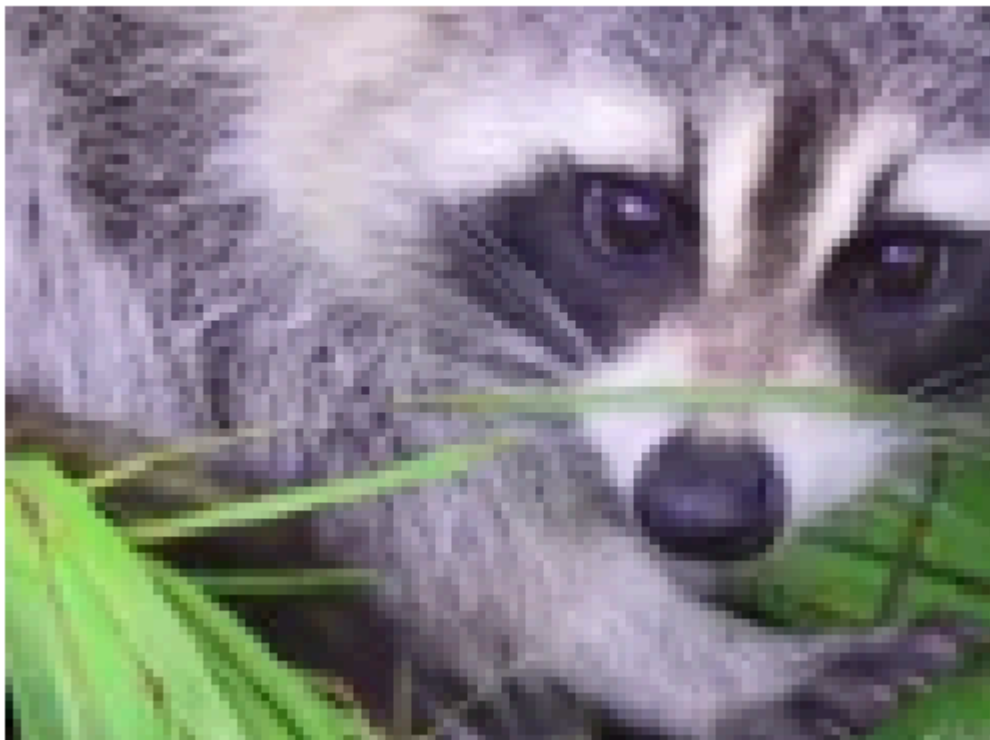
```
import imageio.v3 as iio
import matplotlib.pyplot as plt
```

```
# Tasvirni o'qiyviz
img = iio.imread("images/raccoon.png")
h, w, _ = img.shape

# Markaziy qismni qirqib olamiz
# h//4 dan 3*h//4 gacha bo'lgan qatorlar va w//4 dan 3*w//4 gacha bo'lg
crop = img[h//4 : 3*h//4, w//4 : 3*w//4]

plt.imshow(crop)
plt.axis('off')
plt.title("Qirqilgan Yenot")
plt.show()
```

Qirqilgan Yenot



Izoh:

- `img.shape` : Tasvir o'lchamlarini beradi (balandlik `h`, kenglik `w`, rang kanallari `_`).
- `img[h//4 : 3*h//4, w//4 : 3*w//4]` : Massivni kesish orqali markaziy sohani tanlab oladi. Bu yerda biz balandlik va kenglikning o'rtadagi 50% qismini saqlab qoldik.
- `plt.imshow()` : Qirqilgan qismni vizuallashtiradi.

Muhim eslatma: Kodda `x, y, _ = img.shape` deb yozilgan bo'lsa, kesish qismida ham `x` va `y` o'zgaruvchilaridan foydalanish kerak. Yuqoridagi misolda biz tushunarli bo'lishi uchun `h` (height) va `w` (width) belgilashlarini

ishlatdik. NumPy-da birinchi indeks har doim qatorlar (balandlik), ikkinchisi esa ustunlar (kenglik) hisoblanadi.

Tasvirni aylantirish (Vertikal - Flipping)

Tasvirni aylantirish (tepa-pastga yoki chap-o'ngga) tasvirga ishlov berish va sun'iy intellekt modellarini o'qitishda ma'lumotlarni ko'paytirish (**data augmentation**) uchun keng qo'llaniladigan usuldir.

```
import imageio.v3 as iio
import matplotlib.pyplot as plt
import numpy as np

# Tasvirni o'qiyviz
img = iio.imread("images/raccoon.png")

# Tasvirni tepadan pastga (vertikal) aylantiramiz
flipped = np.flipud(img)

plt.imshow(flipped)
plt.axis('off')
plt.title("Aylantirilgan tasvir (Tepa-pastga)")
plt.show()
```

Aylantirilgan tasvir (Tepa-pastga)



Izoh:

- `np.flipud(img)` : Tasvirni vertikal o‘q bo‘ylab aylantiradi (flip up-down). Bunda massivning qatorlari teskari tartibda joylashadi.
- **Qo‘shimcha:** Agar tasvirni chapdan o‘ngga (gorizontal) aylantirmoqchi bo‘lsangiz, `np.fliplr(img)` (flip left-right) funksiyasidan foydalanishingiz mumkin.

Ushbu amallar NumPy massivining ko‘rsatkichlarini (indexing) o‘zgartirish orqali amalga oshiriladi, shuning uchun ular juda tez va samarali ishlaydi.

Tasvirlarni filtrlash

Filtrlash — tasvirga ishlov berishning fundamental usuli bo‘lib, tasvirdagi muayyan xususiyatlarni kuchaytirish yoki kuchsizlantirish uchun ishlatiladi. Bu usul tasvirni tekislash (**smoothing**), aniqlashtirish (**sharpening**) va chegaralarni aniqlash (**edge detection**) kabi vazifalarda yordam beradi.

1. Gaussian Blur (Gaus bo‘yicha xiralashtirish)

Xiralashtirish tasvirdagi shovqinlarni (noise) va mayda detallarni kamaytirish uchun Gaus yadrosidan (Gaussian kernel) foydalanadi. Bu ko‘pincha chegaralarni aniqlash yoki chegaraviy qiymatni belgilash (thresholding) kabi bosqichlardan oldin tayyorgarlik vazifasini o‘taydi.

```
from scipy.ndimage import gaussian_filter
import imageio.v3 as iio
import matplotlib.pyplot as plt
import numpy as np

# Tasvirni o‘qiyamiz
img = iio.imread("images/raccoon.png")

# Gaus filtrini qo‘llaymiz
# sigma=5 xiralashtirish darajasini belgilaydi
blurred = gaussian_filter(img, sigma=5)

# Natijani ko‘rsatishda ma’lumot turini uint8 ga o‘tkazamiz
plt.imshow(blurred.astype(np.uint8))
plt.axis('off')
plt.title("Gaus bo‘yicha xiralashtirilgan")
plt.show()
```


Gaus bo'yicha xiralashtirilgan



Izoh:

- `gaussian_filter(img, sigma=5)` : Tasvirni Gaus yadrosi yordamida tekislaydi (xiralashtiradi).
- `sigma` : Xiralashtirish intensivligini boshqaradi. `sigma` qancha katta bo'lsa, tasvir shunchalik xira bo'ladi.
- `.astype(np.uint8)` : Filtrlash natijasida olingan suzuvchi nuqtali (float) sonlarni qaytadan butun sonlarga o'tkazadi. Bu ranglarning to'g'ri ko'rinishini ta'minlash uchun muhimdir.

2. Tasvirni aniqlashtirish (Unsharp Masking)

Tasvirni aniqlashtirish (sharpening) detallar va aniqlikni oshirish uchun chegaralar orasidagi kontrastni kuchaytiradi. **Unsharp masking** usuli tasvirning asl nusxasidan uning xiralashtirilgan nusxasini ayirish orqali ishlaydi, bu esa "detallar niqobi"ni hosil qiladi.

```
#!/pip install scikit-image
from skimage.color import rgb2gray, rgba2rgb
from scipy.ndimage import gaussian_filter
import imageio.v3 as iio
import matplotlib.pyplot as plt
import numpy as np
```

```

# Tasvirni o'qiyamiz
img = io.imread("images/raccoon.png")

# Agar tasvirda shaffoflik kanali (Alpha) bo'lsa, uni RGB ga o'tkazamiz
if img.shape[-1] == 4:
    img = rgba2rgb(img)

# Tasvirni kulrang (grayscale) formatga o'tkazamiz va float tipiga keltiramiz
gray = rgb2gray(img).astype(float)

# Xiralashtirilgan nusxasini yaratamiz
blur = gaussian_filter(gray, 5)

# Kuchaytirish ko'effitsienti
alpha = 30

# Unsharp Masking formulasi:  $Asl + alpha * (Asl - Xira)$ 
sharp = gray + alpha * (gray - gaussian_filter(blur, 1))

plt.imshow(sharp, cmap='gray')
plt.axis('off')
plt.title("Aniqlashtirilgan tasvir")
plt.show()

```

Aniqlashtirilgan tasvir



Izoh:

- **rgb2gray** : Rangli tasvirni kulrang ko‘rinishga o‘tkazadi, bu aniqlashtirish algoritmi uchun hisoblashni osonlashtiradi.
- **gray - gaussian_filter(blur, 1)** : Bu amal tasvirdagi chegaralar va detallarni (yuqori chastotali komponentlarni) ajratib oladi.
- **alpha** : Aniqlashtirish darajasini belgilaydi. Bu qiymat qanchalik katta bo‘lsa, chegaralar shunchalik keskinlashadi.
- **Unsharp Masking**: "Xira bo‘lmagan niqoblash" deb atalishiga sabab — asl tasvirga detallar qo‘shilishidan oldin xira nusxadan foydalanib detal qidirilishidir.

Tasvirlarni shovqindan tozalash (Denoising)

Tasvirlarni shovqindan tozalash — bu tasvir sifatini oshirish uchun undagi tasodifiy xatoliklarni (shovqinlarni) olib tashlash jarayonidir. Bu, ayniqsa, past yorug‘likda olingan fotosuratlar yoki skanerlangan hujjatlar bilan ishlashda juda muhim.

1. Shovqin qo‘shish

Denoising algoritmlarini sinab ko‘rish uchun avval sun'iy ravishda shovqin qo‘shiladi. Bu real hayotdagi past yorug‘lik datchiklari yoki sensorlarning kamchiliklari natijasida yuzaga keladigan shovqinli muhitni simulyatsiya qiladi.

```
import numpy as np
import imageio.v3 as iio
import matplotlib.pyplot as plt
from skimage.color import rgb2gray, rgba2rgb

# Tasvirni o‘qiyamiz
img = iio.imread("images/raccoon.png")

# Agar tasvirda shaffoflik kanali bo‘lsa, uni RGB ga o‘tkazamiz
if img.shape[-1] == 4:
    img = rgba2rgb(img)

# Tasvirni kulrang (grayscale) formatga o‘tkazamiz
gray = rgb2gray(img).astype(float)

# Tasvirga tasodifiy shovqin qo‘shish
# 0.9 * gray.std() shovqinning kuchini belgilaydi
noise_img = gray + 0.9 * gray.std() * np.random.random(gray.shape)
```

```
plt.imshow(noise_img, cmap='gray')
plt.axis('off')
plt.title("Shovqinli tasvir")
plt.show()
```

Shovqinli tasvir



Izoh:

- `np.random.random(gray.shape)` : Tasvir bilan bir xil o'lchamdagi tasodifiy sonlar massivini yaratadi.
- `gray.std()` : Tasvirning standart og'ishi yordamida shovqinni masshtablaydi. Bu shovqinning tasvir darajasiga mutanosib bo'lishini ta'minlaydi.
- **Maqsad:** Shovqinli tasvirni yaratish orqali keyingi bosqichda turli filtrlarni (masalan, Median yoki Wiener filtrini) qo'llab, ularning qay darajada samarali ekanligini tekshirish mumkin.

Siz ushbu "shovqinli" tasvirni olganingizdan so'ng, uni tozalash uchun qandaydir filtr qo'llashni rejalashtiryapsizmi (masalan, `median_filter`)?

2. Gaus usuli bilan shovqindan tozalash (Gaussian Denoising)

Gaus filtri piksellar qiymatlarini ularning qo'shnilari bilan Gaus yadrosi (Gaussian kernel) yordamida o'rtachalashtiradi. Bu usul yuqori chastotali

shovqinlarni samarali ravishda kamaytiradi va tasvirni tekislaydi.

```
from scipy.ndimage import gaussian_filter
import matplotlib.pyplot as plt

# Shovqinli tasvirga Gaus filtrini qo'llaymiz
# sigma=2.2 shovqinni tozalash intensivligini belgilaydi
denoised = gaussian_filter(noise_img, sigma=2.2)

plt.imshow(denoised, cmap='gray')
plt.axis('off')
plt.title("Shovqindan tozalangan (Gaussian)")
plt.show()
```

Shovqindan tozalangan (Gaussian)



Izoh:

- `gaussian_filter(noise_img, sigma=2.2)` : Gaus yadrosi yordamida tasvirni tekislaydi va yuqori chastotali tasodifiy nuqtalarni (shovqinni) yo'qotadi.
- **Tuzilmani saqlash:** Gaus filtri shovqinni yaxshi kamaytirsada, u tasvirning keskin chegaralarini (edges) ham biroz yumshatib yuborishi mumkin. `sigma` qiymatini tanlashda ehtiyot bo'lish kerak: juda katta qiymat tasvirni haddan tashqari xira qilib yuboradi.

Maslahat: Agar siz tasvir chegaralarini (masalan, ob'ektlar qirralarini) saqlab qolgan holda shovqinni tozalashni istasangiz, ko'pincha **Median filtri** (`scipy.ndimage.median_filter`) Gaus filtriga qaraganda samaraliroq hisoblanadi, chunki u chegaralarni xiralashtirmaydi.

Ushbu tozalash natijasi sizni qoniqtiradimi yoki chegaralarni saqlab qoluvchi boshqa filtrlarni ham ko'rib chiqamizmi?

Sobel filtri yordamida chegaralarni aniqlash (Edge Detection)

Sobel filtri — bu tasvirdagi piksellar intensivligining gradientini (o'zgarish darajasini) hisoblash orqali ob'ektlarning chegaralarini aniqlash usuli. U 3×3 o'lchamdagi maxsus yadrolar (kernels) yordamida gorizonta va vertikal o'zgarishlarni aniqlaydi va ularni birlashtirib, tasvirdagi chegaralarni ajratib ko'rsatadi. Bu usul ob'ektlarni aniqlash va segmentatsiya qilish kabi vazifalarda juda muhimdir.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import rotate, gaussian_filter, sobel

# 1. Sintetik (sun'iy) tasvir yaratamiz
im = np.zeros((300, 300))
im[64:-64, 64:-64] = 1 # Markazda oq kvadrat chizamiz

# Tasvirni 30 darajaga aylantiramiz va biroz xiralashtiramiz
im = rotate(im, 30, mode='constant')
im = gaussian_filter(im, sigma=7)

plt.imshow(im, cmap='gray')
plt.axis('off')
plt.title("Asl sintetik tasvir")
plt.show()

# 2. Sobel filtrini qo'llaymiz
dx = sobel(im, axis=0, mode='constant') # Vertikal chegaralar (x-o'qi bo'yicha)
dy = sobel(im, axis=1, mode='constant') # Gorizonta chegaralar (y-o'qi bo'yicha)

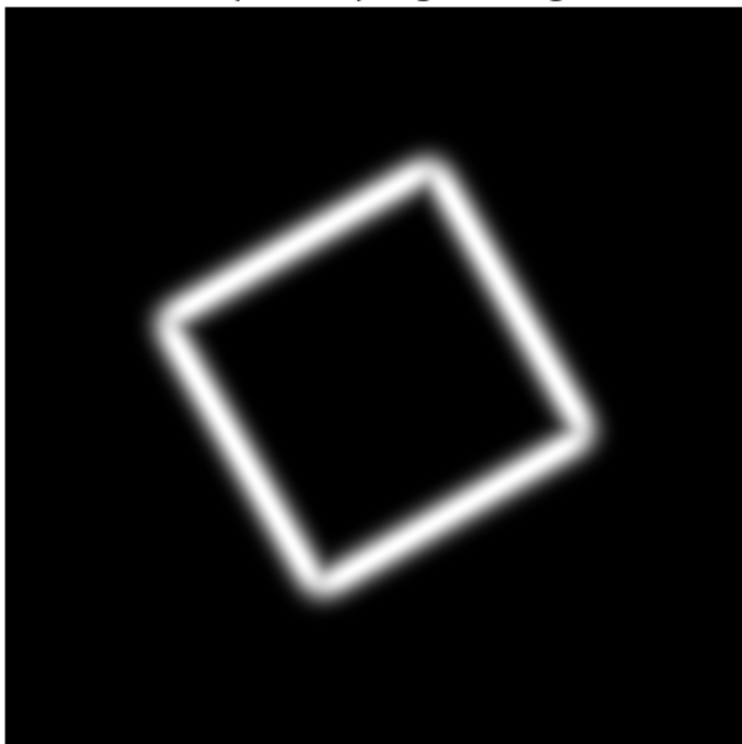
# Ikki gradientni birlashtiramiz (Gipotenuza formulasi orqali)
sobel_edges = np.hypot(dx, dy)

plt.imshow(sobel_edges, cmap='gray')
plt.axis('off')
plt.title("Sobel orqali aniqlangan chegaralar")
plt.show()
```

Asl sintetik tasvir



Sobel orqali aniqlangan chegaralar



Kodning ishlash prinsipi:

- `sobel(im, axis=0)` va `axis=1` : Bu funksiyalar tasvir bo‘ylab rang o‘zgarishi eng yuqori bo‘lgan joylarni qidiradi. Masalan, qora fondan oq kvadratga o‘tilgan joyda gradient qiymati yuqori bo‘ladi.

- `np.hypot(dx, dy)` : Gorizonta (dx) va vertikal (dy) o'zgarishlarni $\sqrt{dx^2 + dy^2}$ formulasi bo'yicha birlashtiradi. Bu chegaralarning umumiy "kuchini" (intensivligini) beradi.
- **Gaussian Blur**: Sobel filtrini qo'llashdan oldin Gaus xiralashtirishini ishlatish juda muhim, chunki u tasvirdagi mayda shovqinlarni yo'qotadi. Aks holda, filtr har bir mayda nuqtani "chegara" deb qabul qilishi mumkin edi.

Sobel filtri chegaralarni qalinroq qilib ko'rsatadi. Agar sizga juda ingichka va aniq chegaralar kerak bo'lsa, kelajakda **Canny Edge Detector** algoritmini ham ko'rib chiqishingiz mumkin.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. Tasvirlarga ishlov berishda filtrlash texnikasi asosan qaysi maqsadlarda qo'llaniladi?
2. `np.uint8` ma'lumot turi tasvirlarni ko'rsatishda (display) nima uchun muhim ahamiyatga ega?
3. Tasvirlarga sun'iy shovqin qo'shishdan ko'zlangan asosiy maqsad nima?
4. Sobel filtri tasvirdagi chegaralarni aniqlashda qaysi matematik tushunchaga (gradient) asoslanadi?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `gaussian_filter()` funksiyasidagi `sigma` parametri tasvirning xiralashish darajasiga qanday ta'sir qiladi?
6. Unsharp Masking usulida xiralashtirilgan nusxa asl tasvirdan ayirilganda qanday ma'lumotlar hosil bo'ladi?
7. Shovqinli tasvirni tozalashda (`denoising`) Gaus filtri yuqori chastotali shovqinlarni qanday yo'qotadi?
8. Sobel operatorida `axis=0` va `axis=1` parametrlari chegaralarni qaysi yo'nalishlar bo'yicha qidiradi?

3-daraja: Amaliy tahlil va murakkab algoritmlar

9. Tasvirni aniqlashtirishda (`sharpening`) `alpha` koeffitsientining ortishi natijaviy tasvir sifatiga qanday ta'sir ko'rsatadi?

10. Nima uchun Sobel filtrini qo'llashdan oldin Gaus xiralashtirishidan foydalanish tavsiya etiladi?
11. `np.hypot(dx, dy)` funksiyasi gorizontal va vertikal gradientlarni birlashtirib, chegaralarning intensivligini qanday hisoblaydi?
12. Tasvirdagi obyektlarni segmentatsiya qilishda chegaralarni aniqlash texnikasining o'rni va ahamiyatini izohlang.